Skill Week-4

Session-7

Task 1: Process Synchronization and Child Process Monitoring Using waitpid()

```
teja@LAPTOP-FGDPB110:~$ mkdir -p ~/OS_Skills
teja@LAPTOP-FGDPB110:~$ cd ~/OS_Skills
teja@LAPTOP-FGDPB110:~/OS_Skills$ nano waitpid_demo.c
teja@LAPTOP-FGDPB110:~/OS_Skills$ gcc waitpid_demo.c -o waitpid_demo
teja@LAPTOP-FGDPB110:~/OS_Skills$ ./waitpid_demo
Child 1: PID = 1279
Parent: PID = 1278
Parent is waiting for child processes...
Child 2: PID = 1280
Child 2: Exiting
Child 1: Exiting
Parent: Child 1 exited with status 10
Parent: Child 2 exited with status 20
Parent: All child processes completed.
teja@LAPTOP-FGDPB110:~/OS_Skills$
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t child1, child2;
    int status;

    child1 = fork();

    if (child1 < 0) {
        perror("fork");
        return 1;
    }

    if (child1 == 0) {
        printf("Child 1: PID = %d\n", getpid());
        sleep(2);
        printf("Child 1: Exiting\n");
        exit(10);
    }

    child2 = fork();

    if (child2 < 0) {
        perror("fork");
        return 1;
    }

    if (child2 == 0) {
        printf("Child 2: PID = %d\n", getpid());
        sleep(1);
        printf("Child 2: Exiting\n");
        exit(20);
    }

    printf("Parent: PID = %d\n", getpid());
    printf("Parent is waiting for child processes...\n");

    pid_t finished = waitpid(child1, &status, 0);

    if (finished == -1) {
        perror("waitpid");
        return 1;
    }

    if (WIFEXITED(status)) {
        printf("Parent: Child 1 exited with status %d\n",
               WEXITSTATUS(status));
    }

    finished = waitpid(child2, &status, 0);

    if (finished == -1) {
        perror("waitpid");
        return 1;
    }

    if (WIFEXITED(status)) {
        printf("Parent: Child 2 exited with status %d\n",
               WEXITSTATUS(status));
    }

    printf("Parent: All child processes completed.\n");

    return 0;
}
```

Task 2: To Retrieve PATH Variable, Parse Search Directories, Locate Executables, Verify Permissions, Handle Missing Commands, Test Command Resolution

```
teja@LAPTOP-FGDPB110       :~/OS_Skills$ nano path_search.c
teja@LAPTOP-FGDPB110       :~/OS_Skills$ gcc path_search.c -o path_search
teja@LAPTOP-FGDPB110       :~/OS_Skills$ ./path_search ls
PATH variable:
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/u
sr/lib/wsl/lib:/mnt/c/WINDOWS/system32:/mnt/c/WINDOWS:/mnt/c/WINDOWS/System32/Wbem:/mnt/c/W
INDOWS/System32/WindowsPowerShell/v1.0/:/mnt/c/WINDOWS/System32/OpenSSH/:/mnt/c/Program Fil
es (x86)/NVIDIA Corporation/PhysX/Common:/mnt/c/Program Files/NVIDIA Corporation/NVIDIA NvD
LISR:/mnt/c/Program Files/nodejs/:/mnt/c/Program Files/MySQL/MySQL Shell 8.0/bin/:/mnt/c/Us
ers/JXIN/AppData/Local/Microsoft/WindowsApps:/mnt/c/Users/JXIN/AppData/Local/Programs/Micro
soft VS Code/bin:/mnt/c/Users/JXIN/AppData/Roaming/npm:/snap/bin

Checking: /usr/local/sbin/ls
Checking: /usr/local/bin/ls
Checking: /usr/sbin/ls
Checking: /usr/bin/ls

Command found!
Executable path: /usr/bin/ls
teja@LAPTOP-FGDPB110       :~/OS_Skills$ ./path_search gcc
PATH variable:
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/u
sr/lib/wsl/lib:/mnt/c/WINDOWS/system32:/mnt/c/WINDOWS:/mnt/c/WINDOWS/System32/Wbem:/mnt/c/W
INDOWS/System32/WindowsPowerShell/v1.0/:/mnt/c/WINDOWS/System32/OpenSSH/:/mnt/c/Program Fil
es (x86)/NVIDIA Corporation/PhysX/Common:/mnt/c/Program Files/NVIDIA Corporation/NVIDIA NvD
LISR:/mnt/c/Program Files/nodejs/:/mnt/c/Program Files/MySQL/MySQL Shell 8.0/bin/:/mnt/c/Us
ers/JXIN/AppData/Local/Microsoft/WindowsApps:/mnt/c/Users/JXIN/AppData/Local/Programs/Micro
soft VS Code/bin:/mnt/c/Users/JXIN/AppData/Roaming/npm:/snap/bin

Checking: /usr/local/sbin/gcc
Checking: /usr/local/bin/gcc
Checking: /usr/sbin/gcc
Checking: /usr/bin/gcc

Command found!
Executable path: /usr/bin/gcc
teja@LAPTOP-FGDPB110       :~/OS_Skills$
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/stat.h>

int main(int argc, char *argv[]) {

    if (argc != 2) {
        printf("Usage: %s <command>\n", argv[0]);
        return 1;
    }

    char *path = getenv("PATH");

    if (path == NULL) {
        printf("PATH variable not found.\n");
        return 1;
    }

    printf("PATH variable:\n%s\n\n", path);

    char *path_copy = strdup(path);

    if (path_copy == NULL) {
        perror("strdup");
        return 1;
    }

    char *dir = strtok(path_copy, ":");

    while (dir != NULL) {

        char full_path[1024];

        snprintf(full_path, sizeof(full_path),
                 "%s/%s", dir, argv[1]);

        printf("Checking: %s\n", full_path);

        struct stat file_info;

        if (stat(full_path, &file_info) == 0) {

            if (S_ISREG(file_info.st_mode) &&
                access(full_path, X_OK) == 0) {
```

```
                printf("\nCommand found!\n");
                printf("Executable path: %s\n", full_path);

                free(path_copy);
                return 0;
            }
        }

        dir = strtok(NULL, ":");
    }

    printf("\nCommand not found or not executable: %s\n",
           argv[1]);

    free(path_copy);

    return 1;
}
```

Session-8

Task 1: To Detect Variable References, Expand Values, Handle Undefined Variables, Update Tokens, Support Nested Variables, Test Expansion Logic

```
teja@LAPTOP-FGDPB110     :~/OS_Skills$ nano variable_expand.c
teja@LAPTOP-FGDPB110     :~/OS_Skills$ gcc variable_expand.c -o variable_expand
teja@LAPTOP-FGDPB110     :~/OS_Skills$ ./variable_expand
Enter a string: Hello $USER
Expanded string: Hello teja
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_SIZE 2048

void expand_variables(const char *input, char *output) {

    int i = 0;
    int j = 0;

    while (input[i] != '\0' && j < MAX_SIZE - 1) {

        if (input[i] == '$') {

            i++;

            char variable[100];
            int k = 0;

            /* Handle ${VARIABLE} */
            if (input[i] == '{') {
                i++;

                while (input[i] != '\0' &&
                       input[i] != '}' &&
                       k < 99) {

                    variable[k++] = input[i++];
                }

                if (input[i] == '}')
                    i++;

            }

            /* Handle $VARIABLE */
            else {
                while (input[i] != '\0' &&
                       (isalnum(input[i]) || input[i] == '_') &&
                       k < 99) {

                    variable[k++] = input[i++];
                }
            }
```

```
            variable[k] = '\0';

            if (k > 0) {

                char *value = getenv(variable);

                if (value != NULL) {

                    int len = strlen(value);

                    if (j + len < MAX_SIZE - 1) {
                        strcpy(&output[j], value);
                        j += len;
                    }

                } else {
                    printf("Warning: Variable '%s' is undefined.\n",
                           variable);
                }

            } else {
                output[j++] = '$';
            }

        } else {

            output[j++] = input[i++];
        }
    }

    output[j] = '\0';
}

int main() {

    char input[MAX_SIZE];
    char output[MAX_SIZE];

    printf("Enter a string: ");

    if (fgets(input, sizeof(input), stdin) == NULL)
        return 1;

    input[strcspn(input, "\n")] = '\0';

    expand_variables(input, output);

    printf("Expanded string: %s\n", output);

    return 0;
}
```

Task 2: To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic

```
teja@LAPTOP-FGDPB110     :~/OS_Skills$ nano builtin_shell.c
teja@LAPTOP-FGDPB110     :~/OS_Skills$ gcc builtin_shell.c -o builtin_shell
teja@LAPTOP-FGDPB110     :~/OS_Skills$ ./builtin_shell
Simple Shell
Built-ins: cd, pwd, echo, exit
myshell> pwd
/home/teja/OS_Skills
myshell> echo Hello Ubuntu
Hello Ubuntu
myshell> cd /tmp
myshell> pwd
/tmp
myshell> hello
Command not found or not a built-in: hello
myshell> exit
Exiting shell...
teja@LAPTOP-FGDPB110     :~/OS_Skills$
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define MAX_INPUT 1024
#define MAX_ARGS 100

/* Built-in command functions */

int builtin_pwd(char **args) {
    char cwd[1024];

    if (getcwd(cwd, sizeof(cwd)) != NULL) {
        printf("%s\n", cwd);
        return 0;
    }

    perror("pwd");
    return 1;
}

int builtin_cd(char **args) {

    if (args[1] == NULL) {

        char *home = getenv("HOME");

        if (home == NULL) {
            printf("HOME variable not found.\n");
            return 1;
        }

        if (chdir(home) != 0) {
            perror("cd");
            return 1;
        }

    } else {

        if (chdir(args[1]) != 0) {
            perror("cd");
            return 1;
        }
    }
```

```c
    return 0;
}

int builtin_echo(char **args) {

    int i = 1;

    while (args[i] != NULL) {
        printf("%s", args[i]);

        if (args[i + 1] != NULL)
            printf(" ");

        i++;
    }

    printf("\n");

    return 0;
}

int builtin_exit(char **args) {

    printf("Exiting shell...\n");

    exit(0);
}

/* Dispatch table */

struct builtin {
    char *name;
    int (*function)(char **args);
};

struct builtin builtin_table[] = {
    {"pwd", builtin_pwd},
    {"cd", builtin_cd},
    {"echo", builtin_echo},
    {"exit", builtin_exit}
};

int builtin_count =
    sizeof(builtin_table) / sizeof(builtin_table[0]);
```

```c
int execute_builtin(char **args) {

    if (args[0] == NULL)
        return 1;

    for (int i = 0; i < builtin_count; i++) {

        if (strcmp(args[0], builtin_table[i].name) == 0) {
            return builtin_table[i].function(args);
        }
    }

    return -1;
}

/* Parse input */

void parse_input(char *input, char **args) {

    int i = 0;

    args[i] = strtok(input, " \t\n");

    while (args[i] != NULL && i < MAX_ARGS - 1) {

        i++;
        args[i] = strtok(NULL, " \t\n");
    }

    args[i] = NULL;
}

int main() {

    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    printf("Simple Shell\n");
    printf("Built-ins: cd, pwd, echo, exit\n");

    while (1) {

        printf("myshell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
```

```c
        if (input[0] == '\n')
            continue;

        parse_input(input, args);

        int result = execute_builtin(args);

        if (result == -1) {
            printf("Command not found or not a built-in: %s\n",
                   args[0]);
        }
    }

    return 0;
}
```